Blackboard Exercise 3: Worked solutions

Algorithms, Programming, and Data Representation · Module 92313 · Blackboard Exercise 3

From Fundamentals to AI Apocalypse · personal study guide

Contents

Exercise 3.1: Memory chip architecture 2

Exercise 3.2: Memory modules architecture 3

Exercise 3.3: Byte-addressable memory modules 3

Exercise 3.4: Endianness bytes 4

Answers 5

Made by Thanate Amornjiraporn (Focus)

Work through memory size, addressing, module construction, interleaving and byte order. For address calculations, state whether an address selects a word or a byte.

Exercise 3.1: Memory chip architecture

Connected topic: Addressing and capacity (§5.2)

3.1.1 Memory size in bits

Try first: For a 4G × 32-bit memory, find the total capacity in bits and write it as 2^x.

Answers: see "Exercise 3.1: Memory chip architecture" under Answers at the end.

3.1.2 Word-addressable memory

Try first: Find the address width when each address selects one 32-bit word.

Answers: see "Exercise 3.1: Memory chip architecture" under Answers at the end.

3.1.3 Byte-addressable memory

Try first: Find the address width when each address selects one byte.

Answers: see "Exercise 3.1: Memory chip architecture" under Answers at the end.

3.1.4 Byte address range

Try first: Give the lowest and highest byte addresses in hexadecimal and the number of bits needed to represent them.

Answers: see "Exercise 3.1: Memory chip architecture" under Answers at the end.

Made by Thanato Amornjirapon (Focus)

Exercise 3.2: Memory modules architecture

Connected topic: Memory chips and modules (§5.4) · Memory interleaving (§5.5)

3.2.1 Module configuration

Try first: Complete the chip and address-width specifications for a word-addressable 1G × 32-bit memory built from 256M × 4-bit chips.

Answers: see "Exercise 3.2: Memory modules architecture" under Answers at the end.

3.2.2 Word 14 across four modules

Try first: Locate word address 14 under high-order and low-order interleaving, numbering modules from zero.

Answers: see "Exercise 3.2: Memory modules architecture" under Answers at the end.

3.2.3 Word 300,000,000

Try first: Find the module and local word address for word address 300,000,000 under both interleaving schemes.

Answers: see "Exercise 3.2: Memory modules architecture" under Answers at the end.

Exercise 3.3: Byte-addressable memory modules

Connected topic: Addressing and capacity (§5.2) · Memory interleaving (§5.5)

3.3.1 Full-memory address width

Try first: For the same 1G × 32-bit memory, find the number of address bits when it is byte-addressable.

Answers: see "Exercise 3.3: Byte-addressable memory modules" under Answers at the end.

3.3.2 Word 14 after byte-addressing

Try first: Locate word 14 under both interleaving schemes. The memory modules still store complete 32-bit words.

Answers: see "Exercise 3.3: Byte-addressable memory modules" under Answers at the end.

Made by Thanate Amornjitaporn (Focus)

Exercise 3.4: Endianness bytes

Connected topic: Endianness and aligned access (§5.3)

3.4.1 Define endianness

Try first: State the definition of endianness in one sentence.

Answers: see "Exercise 3.4: Endianness bytes" under Answers at the end.

3.4.2 Transfer between different byte orders

Try first: Explain what must happen when data moves from little-endian memory to big-endian memory.

Answers: see "Exercise 3.4: Endianness bytes" under Answers at the end.

3.4.3 Transfer 0x12345678 unchanged

Try first: A 32-bit integer 0x12345678 is stored little-endian and copied unchanged into big-endian memory. Find the value read there and decide whether the original can be recovered without byte-order information.

Answers: see "Exercise 3.4: Endianness bytes" under Answers at the end.

Made by Thanate Amornjiaporn (Focus)

Answers

Exercise 3.1: Memory chip architecture

3.1.1 Memory size in bits · worked solution

1. In the notation 4G × 32 the first number counts the words and the second gives the bits in each word. In memory sizes G means 2^{30}, so $4\text{G} = 2^2 \cdot 2^{30} = 2^{32}$ words.
2. Capacity is words times bits per word:

$$2^{32} \times 32 = 2^{32} \times 2^5 = 2^{37}\,\text{bits}$$

   So $\boldsymbol{x = 37}$.
3. **Check** the units: words × bits/word = bits. A sketch helps: a column of 2^{32} rows, each row 32 bits wide.

Exercise 3.1: Memory chip architecture

3.1.2 Word-addressable memory · worked solution

1. Word-addressable means every 32-bit word gets its own address, so we need one address for each of the 2^{32} words.
2. A k-bit address can name 2^k places:

$$\log_2(2^{32}) = 32\,\text{address bits}$$

3. **Check:** 32 bits give the addresses 0 to $2^{32} - 1$, exactly one per word.

Exercise 3.1: Memory chip architecture

3.1.3 Byte-addressable memory · worked solution

1. Byte-addressable means every byte gets its own address. Each 32-bit word holds 4 bytes:

$$2^{32}\,\text{words} \times 4\,\text{bytes} = 2^{32} \times 2^2 = 2^{34}\,\text{bytes}$$

2. Naming 2^{34} bytes takes **34 address bits**.
3. **Check:** two bits more than word addressing, because each word now has $2^2 = 4$ byte addresses inside it.

Exercise 3.1: Memory chip architecture

3.1.4 Byte address range · worked solution

1. The 2^{34} byte addresses are numbered from 0, so the highest is one less than the count.
2. $$\text{lowest} = 0\text{x}000000000$$
$$\text{highest} = 2^{34} - 1 = 0\text{x}3\text{FFFFFFFF}$$
Writing them takes **34 bits**: $2^{34} - 1$ is thirty-four 1s, which is 3 followed by eight F digits in hexadecimal ($2 + 8 \cdot 4 = 34$ bits).
3. **Check:** the next address, $0\text{x}400000000 = 2^{34}$, lies outside the memory. The leading zeros of the lowest address only pad it to the same width.

Exercise 3.2: Memory modules architecture

3.2.1 Module configuration · worked solution

1. Build the word width first. One chip delivers 4 bits per word, so 32 / 4 = **8 chips** side by side form a 32-bit module. A module then holds as many words as one chip: 256M = 2^{28}.
2. Then build the word count. The memory needs 1G = 2^{30} words:
$$\text{modules} = \frac{2^{30}}{2^{28}} = 4$$
$$\text{chips} = 4 \times 8 = 32$$
3. Address widths: a module has 2^{28} words, so it needs 28 address bits; the whole memory has 2^{30} words, so 30.

Task	Specification	Quantity
a)	RAM chips per memory module	8
b)	Memory modules	4
c)	RAM chips for the full memory	32
d)	Address bits needed for a memory module	28
e)	Address bits needed for the full memory	30

4. **Check** width and count separately: 8 chips × 4 bits = 32 bits per word, and 4 modules × 256M words = 1G words. The 2 extra address bits for the full memory pick one of the $4 = 2^2$ modules.

Exercise 3.2: Memory modules architecture

3.2.2 Word 14 across four modules · worked solution

1. The memory has $4 = 2^2$ modules, so 2 of the 30 word-address bits select a module and the other 28 select the word inside it. Word 14 is 1110_2, that is 0 … 0 1110 in 30 bits.

2. **High-order interleave** uses the top 2 bits for the module:

$$\text{module} = \lfloor 14 \,/\, 2^{28} \rfloor = 0$$

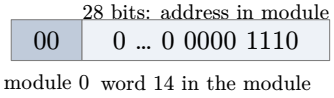
$$\text{word in module} = 14$$

   Low-order interleave uses the bottom 2 bits:

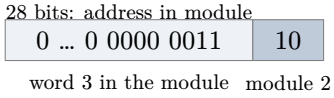
$$\text{module} = 14 \bmod 4 = 2$$

$$\text{word in module} = \lfloor 14 \,/\, 4 \rfloor = 3$$

3. The two address splits:



High-order interleave. The top 2 of the 30 address bits pick the module, so consecutive words fill module 0 first.



Low-order interleave. The bottom 2 bits pick the module, so consecutive words go to modules 0, 1, 2, 3, 0, …

4. **Check** by rebuilding the address. High-order: module 0 starts at word 0, so its word 14 is word 14. Low-order: module 2 holds words 2, 6, 10, 14, …, and 14 is its entry number 3 (counting from 0).

Made by '1

Exercise 3.2: Memory modules architecture

3.2.3 Word 300,000,000 · worked solution

1. High-order: divide by the module size $2^{28} = 268\ 435\ 456$. Low-order: divide by the number of modules, 4.
2. **a) High-order:**

$$300\,000\,000 = 1 \times 268\,435\,456 + 31\,564\,544$$

   Module **1**, word **31 564 544** in that module.
3. **b) Low-order:**

$$300\,000\,000 = 75\,000\,000 \times 4 + 0$$

   Module **0**, word **75 000 000** in that module.
4. **Check** by rebuilding: $1 \times 2^{28} + 31\ 564\ 544 = 300\ 000\ 000$, and $4 \times 75\ 000\ 000 + 0 = 300\ 000\ 000$.

Exercise 3.3: Byte-addressable memory modules

3.3.1 Full-memory address width · worked solution

1. The memory still holds 1G words of 4 bytes, but now every byte has an address:

$$1\text{G} \times 4 = 2^{30} \times 2^{2} = 2^{32}\,\text{bytes}$$

2.

Task	Specification	Quantity
a)	Address bits needed for the full memory	32

   Chips and modules do not change: 8 chips per module, 4 modules, 32 chips.
3. **Check:** 30 bits pick the word, as before, and 2 more pick the byte inside it.

Made

Exercise 3.3: Byte-addressable memory modules

3.3.2 Word 14 after byte-addressing · worked solution

1. Word 14 starts at byte address $14 \times 4 = 56 = 111000_2$. In a 32-bit byte address the lowest 2 bits now pick the byte inside the word, and the 30 bits above them are the old word address.
2. Each module is 32 bits wide, so it holds whole words; the module is therefore chosen from the word address, just above the 2 byte bits.

   High-order: the top 2 address bits pick the module. Word 14 is in **module 0**, at byte address 56 inside it (its word 14).

   Low-order: the 2 bits just above the byte offset pick the module:

   $$\text{module} = 14 \bmod 4 = 2$$

   $$\text{word in module} = \lfloor 14 / 4 \rfloor = 3, \text{ starting at byte } 3 \times 4 = 12 \text{ in the module}$$

   So word 14 is in **module 2**, as in 3.2.2; only the addresses are now counted in bytes.
3. **Assumption.** The sheet does not say which bits interleave. If instead the lowest 2 bits of the *byte* address picked the module, the four bytes of word 14 (addresses 56 to 59) would land in modules 0, 1, 2 and 3, each at position 14. A word would then be split across modules, which does not fit modules built as 32-bit words, so the word-address reading above is the one to use; say so in your answer.
4. **Check** with a sketch: byte addresses 0 to 15 hold words 0 to 3 in modules 0 to 3, so $56 = 4 \times 14$ lies in the row of word 14, module $14 \bmod 4 = 2$.

Exercise 3.4: Endianness bytes

3.4.1 Define endianness · worked solution

1. A multi-byte value occupies several byte addresses, so the system needs a rule for which byte comes first.
2. Endianness is the order in which a multi-byte value's bytes are stored in memory: little-endian places the least significant byte at the lowest address, while big-endian places the most significant byte there.
3. This rule affects multi-byte values; a single-byte value has no byte-order choice.

Exercise 3.4: Endianness bytes

3.4.2 Transfer between different byte orders · worked solution

1. A raw byte copy preserves the sequence of bytes. The receiving system then interprets that sequence using its own byte-order rule.
2. To preserve the numeric value, the sender and receiver must agree on a serialized byte order, or one side must reorder the bytes during transfer. If bytes are copied unchanged without conversion, a multi-byte number may be read as a different value.
3. Single-byte text is unaffected. A robust file or network format specifies byte order rather than relying on the sender's native layout.

Exercise 3.4: Endianness bytes

3.4.3 Transfer 0x12345678 unchanged · worked solution

1. In little-endian order, the bytes at increasing addresses are 78, 56, 34, 12. The transfer keeps that sequence unchanged.
2. A big-endian receiver treats the first byte as most significant, so it reads 0x78563412.
3. The original value cannot be determined from those bytes alone without knowing the sender's byte order or other format metadata: the same four bytes can represent different integers under different conventions.

Made by Thanate Amornjiraporn (Focus)

Personal worked solutions based on the APDR Blackboard Exercise 3 sheet, written in my own words; no exercise-sheet prose or figures are reproduced. Not an official FAU publication; check course details against the current materials. Made by **Thanate Amornjiraporn (Focus)**.